

به نام خدا



دانشکده برق ، کامپیوتر و فناوری های پیشرفته

گروه مهندسی کامپیوتر

پروژه درس رایانش ابری

مدل سازی و تحلیل رفتار Swift-like در CloudSim برای
بهبود کارایی مرکز داده

نام و نام خانوادگی: آی سودا حاجی حسینلو

شماره دانشجویی: ۱۴۰۴۶۹۹۱۱۰

استاد: سرکار خانم دکتر شریفی

۲۶ خرداد ۱۴۰۵

تمامی حقوق این اثر متعلق به دانشگاه ارومیه می باشد.

چکیده

با گسترش مراکز داده مقیاس بالا، کنترل ازدحام شبکه به یکی از چالش‌های اساسی در بهبود کارایی سیستم‌های ابری تبدیل شده است. الگوریتم Swift به عنوان یک روش مبتنی بر تأخیر (Delay-based Congestion Control) با هدف کاهش زمان صف‌بندی و بهبود زمان تکمیل جریان‌ها در محیط‌های دیتاسنتری طراحی شده است. در این پروژه، به جای بازسازی سطح بسته (Packet-level) الگوریتم Swift، یک مدل مفهومی الهام‌گرفته از رفتار آن در محیط CloudSim 7.0.1 پیاده‌سازی شد.

در این شبیه‌سازی، دو سناریو شامل حالت پایه (Baseline) و حالت Swift-like مورد مقایسه قرار گرفتند. شاخص‌های ارزیابی شامل میانگین زمان تکمیل وظایف (Average Finish Time) طول کل اجرای سیستم (Makespan) و نرخ خروجی (Throughput) بودند. نتایج نشان دادند که مدل Swift-like منجر به کاهش قابل توجه در زمان تکمیل وظایف و افزایش نرخ خروجی سیستم می‌شود.

لازم به تأکید است که این مطالعه یک ارزیابی مفهومی در سطح کلان سیستم بوده و به عنوان پیاده‌سازی کامل الگوریتم Swift در سطح TCP یا بسته‌های شبکه محسوب نمی‌شود. با این حال، نتایج به دست آمده نشان می‌دهند که رویکردهای مبتنی بر کاهش تأخیر می‌توانند تأثیر مثبتی بر کارایی کلی مراکز داده داشته باشند.

فهرست مطالب

۱	چکیده	۱.۰.۰
۴	مقدمه	۲.۰.۰
۵	معرفی CloudSim	۳.۰.۰
۷	مروری بر مقاله	۴.۰.۰
۷	مسئله اصلی مقاله	۵.۰.۰
۷	ایده اصلی Swift	۶.۰.۰
۸	مکانیزم کنترلی	۷.۰.۰
۸	نقش Pacing	۸.۰.۰
۸	پیاده‌سازی مقاله	۹.۰.۰
۹	مزایا و ویژگی‌های اصلی Swift	۱۰.۰.۰
۹	نتایج گزارش شده در مقاله	۱۱.۰.۰
۹	ارتباط مقاله با پروژه حاضر	۱۲.۰.۰
۱۰	جمع‌بندی این بخش	۱۳.۰.۰
۱۱	کد پیاده‌سازی شده و توضیحات آن	۱۴.۰.۰
۱۶	ساختار کلی برنامه	۱۵.۰.۰
۱۷	تعریف دیتاسنتر	۱۶.۰.۰
۱۸	ایجاد ماشین‌های مجازی	۱۷.۰.۰
۱۸	ایجاد Cloudlet	۱۸.۰.۰
۱۸	اجرای شبیه‌سازی	۱۹.۰.۰
۱۹	محاسبه شاخص‌های عملکرد	۲۰.۰.۰
۲۰	ذخیره نتایج	۲۱.۰.۰
۲۰	تولید نمودارها	۲۲.۰.۰
۲۰	نحوه اجرا و محیط اجرا	۲۳.۰.۰
۲۱	جمع‌بندی این بخش	۲۴.۰.۰
۲۲	خروجی پروژه	۲۵.۰.۰
۲۲	توضیح تصاویر مربوط به محیط CMD	۲۶.۰.۰
۲۵	توضیح نمودارهای خروجی	۲۷.۰.۰
۲۶	توضیح فایل‌های متنی و داده‌ای	۲۸.۰.۰



۲۷	 لینک کد و خروجی ها در گیت هاب
۲۸	 جدول مقایسه نتایج دو سناریو
۲۸	 تحلیل جدول مقایسه
۳۰	 مزایای پیاده سازی با cloudsim
۳۲	 جمع بندی
۳۳	 نتیجه گیری
۳۴	 فهرست منابع

مقدمه

مراکز داده مدرن به عنوان زیرساخت اصلی خدمات ابری، پردازش‌های عظیم و سیستم‌های توزیع شده، نیازمند مکانیزم‌های کارآمد برای مدیریت منابع و کنترل ازدحام شبکه هستند. افزایش تعداد ماشین‌های مجازی، رشد حجم ترافیک داخلی دیتاسنتر و حساسیت بالای کاربردها به تأخیر، اهمیت استفاده از الگوریتم‌های پیشرفته کنترل ازدحام را دوچندان کرده است.

در سال‌های اخیر، الگوریتم‌های مبتنی بر تأخیر به عنوان جایگزینی برای روش‌های سنتی مبتنی بر از دست رفتن بسته (Loss-based) مطرح شده‌اند. یکی از الگوریتم‌های مطرح در این حوزه، Swift است که با استفاده از اندازه‌گیری دقیق تأخیر صف (Queue Delay) و تنظیم پویا پنجره ازدحام، تلاش می‌کند زمان تکمیل جریان‌ها را در محیط‌های دیتاسنتری کاهش دهد.

با توجه به اینکه CloudSim یک شبیه‌ساز در سطح وظیفه و ماشین مجازی است و قابلیت مدل‌سازی مستقیم بسته‌های شبکه یا پروتکل TCP را ندارد، در این پروژه بازسازی کامل Swift در سطح بسته امکان‌پذیر نبود. بنابراین، یک مدل مفهومی (Conceptual Model) از رفتار Swift طراحی شد که اثر کاهش تأخیر را بر شاخص‌های کلان عملکرد سیستم شبیه‌سازی می‌کند.

هدف این مطالعه، بررسی این پرسش است که آیا اعمال یک سیاست الهام‌گرفته از Swift می‌تواند در سطح کلان سیستم، موجب بهبود شاخص‌های عملکردی نظیر میانگین زمان تکمیل وظایف، طول اجرای کل سیستم و نرخ خروجی شود یا خیر.

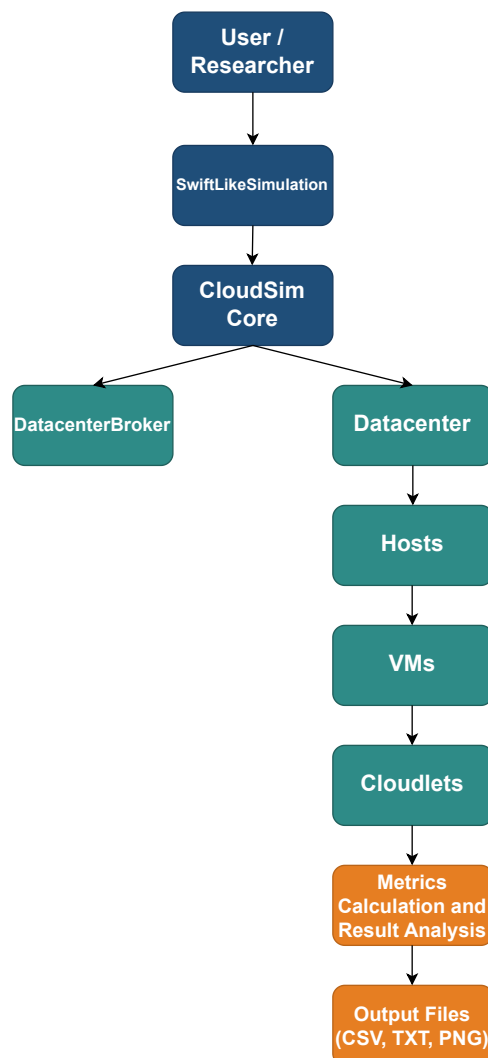
معرفی CloudSim

CloudSim یک چارچوب شبیه‌سازی متن‌باز برای مدل‌سازی و ارزیابی محیط‌های رایانش ابری است که به‌طور گسترده در پژوهش‌های دانشگاهی و صنعتی مورد استفاده قرار می‌گیرد. این ابزار امکان شبیه‌سازی اجزای اصلی یک مرکز داده از جمله میزبان‌ها (Host)، ماشین‌های مجازی (VM)، وظایف محاسباتی (Cloudlet)، زمان‌بندها و سیاست‌های تخصیص منابع را فراهم می‌کند. هدف اصلی CloudSim آن است که پژوهشگر بتواند بدون نیاز به پیاده‌سازی واقعی زیرساخت ابری، رفتار یک سیستم را در سناریوهای مختلف بررسی و تحلیل کند.

در این پروژه، از CloudSim به‌عنوان بستر اصلی برای شبیه‌سازی مفهومی اثر الگوریتم Swift استفاده شده است. از آنجا که Swift در اصل یک الگوریتم کنترل ازدحام در سطح شبکه و TCP است، پیاده‌سازی مستقیم آن در CloudSim ممکن نیست؛ زیرا CloudSim بیشتر بر لایه منابع محاسباتی، ماشین‌های مجازی و زمان‌بندی وظایف تمرکز دارد. با این حال، این چارچوب امکان آن را می‌دهد که اثرات مفهومی بهبود تأخیر و کنترل بهتر جریان داده به‌صورت غیرمستقیم در قالب تغییر پارامترهای اجرایی مانند توان پردازشی ماشین‌های مجازی، پهنای باند و طول وظایف مدل‌سازی شود.

معماری مورد استفاده در این پروژه شامل یک کلاس اصلی برای اجرای شبیه‌سازی، یک بروکر برای مدیریت تخصیص وظایف، یک دیتاسنتر شامل چند میزبان، مجموعه‌ای از ماشین‌های مجازی و تعدادی Cloudlet است. در نهایت، خروجی اجرای هر سناریو به‌صورت معیارهای عملکردی، فایل‌های متنی و نمودارهای مقایسه‌ای ذخیره می‌شود. این ساختار به پژوهشگر اجازه می‌دهد تا دو سناریوی Baseline و Swift-like را در شرایط یکسان مقایسه کرده و اثر تغییرات را بر شاخص‌هایی مانند Average Finish Time، Makespan و Throughput مشاهده کند.

شکل زیر معماری کلی شبیه‌سازی انجام‌شده در این پروژه را نشان می‌دهد. در این ساختار، کاربر یا پژوهشگر از طریق کلاس اصلی SwiftLikeSimulation پارامترهای سناریو را تنظیم و اجرای شبیه‌سازی را آغاز می‌کند. هسته CloudSim وظیفه مدیریت رویدادها و هماهنگی بین اجزای مختلف را بر عهده دارد. DatacenterBroker مسئول تخصیص وظایف به ماشین‌های مجازی است و دیتاسنتر نیز شامل میزبان‌ها و منابع پردازشی مورد نیاز برای اجرای VMها می‌باشد. در نهایت، وظایف محاسباتی در قالب Cloudlet اجرا شده و خروجی حاصل به‌صورت معیارهای عددی، گزارش متنی و نمودارهای گرافیکی ذخیره می‌شود.



شکل ۱: معماری کلی شبیه‌سازی مفهومی الگوریتم Swift در محیط CloudSim

مروری بر مقاله

مقاله Swift: Delay is Simple and Effective for Congestion Control in the Datacenter که در کنفرانس SIGCOMM 2020 منتشر شده است، یکی از کارهای مهم در حوزه کنترل ازدحام در مراکز داده به شمار می‌رود. ایده اصلی این مقاله آن است که تأخیر، در صورتی که به صورت دقیق و سریع اندازه‌گیری شود، می‌تواند یک سیگنال ساده اما بسیار مؤثر برای کنترل ازدحام در شبکه‌های دیتاسنتری باشد. نویسندگان مقاله نشان می‌دهند که به جای اتکا به مکانیزم‌های پیچیده یا وابسته به پیکربندی گسترده سوئیچ‌ها، می‌توان با استفاده از اندازه‌گیری دقیق RTT و واکنش سریع در سمت میزبان، به عملکرد بسیار مناسبی در مقیاس بالا دست یافت.

۱.۰.۰ مسئله اصلی مقاله

در مراکز داده مدرن، برنامه‌های متعددی وجود دارند که نسبت به تأخیر بسیار حساس هستند. برای مثال، سرویس‌های مبتنی بر RPC، سامانه‌های ذخیره‌سازی سریع مانند NVMe و همچنین بارهای کاری توزیع‌شده، نیازمند زمان پاسخ بسیار کم و پایدار هستند. در چنین محیطی، حتی افزایش اندک در تأخیر انتهایی یا Tail Latency می‌تواند باعث افت محسوس عملکرد کل سیستم شود.

نویسندگان مقاله بیان می‌کنند که پروتکل‌های متداول کنترل ازدحام مانند DCTCP اگرچه در بسیاری از شرایط عملکرد مناسبی دارند، اما در سناریوهای شدید مانند Incastهای بزرگ یا بارهای کاری فشرده، ممکن است دچار افزایش محسوس تأخیر و ناپایداری شوند. علاوه بر این، این روش‌ها معمولاً تا حدی به پیکربندی مناسب تجهیزات شبکه وابسته هستند و همین موضوع مدیریت آن‌ها را در شبکه‌های ناهمگن دشوار می‌کند. بر همین اساس، مقاله به دنبال ارائه روشی است که بتواند هم ازدحام در Fabric شبکه و هم ازدحام در Host/NIC را به صورت مؤثر مدیریت کند.

۲.۰.۰ ایده اصلی Swift

ایده محوری Swift این است که RTT می‌تواند به عنوان سیگنال اصلی ازدحام به کار رود. هرگاه تأخیر رفت و برگشت بسته‌ها افزایش پیدا کند، این افزایش می‌تواند نشانه‌ای از تجمع صف، فشار در شبکه، یا ازدحام در نقطه‌های انتهایی باشد. در نتیجه، فرستنده می‌تواند با مشاهده تغییرات تأخیر، نرخ ارسال خود را تنظیم کند.

تفاوت مهم Swift با برخی رویکردهای ساده‌تر مبتنی بر تأخیر در این است که این الگوریتم تنها به مشاهده یک مقدار خام از RTT اکتفا نمی‌کند، بلکه تلاش می‌کند سهم بخش‌های مختلف در ایجاد تأخیر را در نظر بگیرد. در واقع، Swift میان تأخیر ناشی از خود Fabric شبکه و تأخیر ناشی از Endpointها مانند NIC و میزبان تمایز قائل می‌شود. این تفکیک باعث می‌شود کنترل ازدحام صرفاً به شلوغی لینک‌ها محدود نماند و مشکلات ناشی از انتهای ارتباط نیز در تصمیم‌گیری لحاظ شود.

۳.۰.۰ مکانیزم کنترلی

الگوریتم Swift برای تنظیم CWND از یک رویکرد نزدیک به AIMD استفاده می‌کند. هدف آن است که تأخیر مشاهده‌شده در حوالی یک مقدار هدف به نام Target Delay نگه داشته شود. اگر تأخیر کمتر از مقدار هدف باشد، الگوریتم می‌تواند پنجره ازدحام را با احتیاط افزایش دهد تا از ظرفیت شبکه بهتر استفاده شود. در مقابل، اگر تأخیر از حد هدف عبور کند، CWND کاهش پیدا می‌کند تا فشار روی شبکه کم شود و صف‌ها تخلیه شوند.

این روش از آن جهت مهم است که برخلاف پروتکل‌های مبتنی بر Loss، منتظر از دست رفتن بسته نمی‌ماند تا متوجه ازدحام شود. در عوض، با مشاهده زود هنگام افزایش تأخیر، پیش از وقوع افت شدید عملکرد واکنش نشان می‌دهد. همین ویژگی باعث می‌شود Swift در محیط‌های حساس به تأخیر عملکرد بهتری داشته باشد.

۴.۰.۰ نقش Pacing

یکی از ویژگی‌های مهم Swift استفاده از Pacing است. در سناریوهایی که تعداد جریان‌ها زیاد است و مجموع تقاضا از BDP شبکه فراتر می‌رود، صرف کنترل CWND ممکن است کافی نباشد. در چنین شرایطی، Pacing کمک می‌کند بسته‌ها با فاصله زمانی مناسب‌تری ارسال شوند و از تزریق ناگهانی ترافیک به شبکه جلوگیری شود.

این ویژگی به‌ویژه در شرایط Incast اهمیت زیادی دارد؛ یعنی حالتی که تعداد زیادی فرستنده هم‌زمان به یک گیرنده داده ارسال می‌کنند. در چنین وضعیتی، اگر بسته‌ها به‌صورت burst وارد شبکه شوند، صف‌ها به‌سرعت پر می‌شوند و تأخیر به‌شدت افزایش می‌یابد. Pacing این مشکل را تا حد زیادی کنترل می‌کند.

۵.۰.۰ پیاده‌سازی مقاله

مطابق مقاله، Swift در پشته شبکه Pony Express و در معماری Snap پیاده‌سازی شده است. یکی از نکات کلیدی این پیاده‌سازی، استفاده از Hardware Timestamp در NIC برای اندازه‌گیری بسیار دقیق RTT است. این دقت بالا برای موفقیت الگوریتم ضروری است، زیرا Swift باید بتواند تغییرات بسیار کوچک و سریع در تأخیر را تشخیص دهد و بر اساس آن‌ها تصمیم‌گیری کند.

نویسندگان همچنین تأکید می‌کنند که طراحی Swift باید از نظر مصرف پردازنده نیز کارآمد باشد. به همین دلیل، این پروتکل به‌گونه‌ای طراحی شده که در محیط‌های OS-bypass و سامانه‌های با سرعت بالا قابل استفاده باشد. در مراکز داده مدرن، تنها بهبود عملکرد شبکه کافی نیست و سربار پردازشی خود پروتکل نیز باید تا حد امکان پایین نگه داشته شود.

۶.۰.۰ مزایا و ویژگی‌های اصلی Swift

بر اساس مقاله، مهم‌ترین ویژگی‌های Swift را می‌توان به صورت زیر خلاصه کرد:

۱. استفاده از Delay به عنوان سیگنال اصلی و سریع ازدحام؛
۲. واکنش زودهنگام به ازدحام پیش از وقوع Packet Loss؛
۳. تفکیک میان تأخیر ناشی از Fabric و تأخیر ناشی از Endpoint؛
۴. استفاده از AIMD برای تنظیم CWND پیرامون یک Target Delay؛
۵. بهره‌گیری از Pacing برای کنترل بهتر ترافیک در بارهای سنگین؛
۶. مناسب بودن برای شبکه‌های پرسرعت دیتاسنتری با حساسیت بالا نسبت به تأخیر؛
۷. کارایی مناسب از نظر مصرف CPU در پیاده‌سازی‌های واقعی.

۷.۰.۰ نتایج گزارش‌شده در مقاله

نویسندگان مقاله گزارش می‌کنند که Swift در مقایسه با روش‌هایی مانند DCTCP توانسته است Tail Latency را به میزان قابل توجهی کاهش دهد و در عین حال نرخ اتلاف بسته را نیز بهبود بخشد. همچنین مقاله نشان می‌دهد که این پروتکل در نرخ‌های بالا، در حد 100 Gbps به ازای هر سرور، عملکرد پایداری دارد و می‌تواند تعداد بسیار زیادی جریان هم‌زمان را مدیریت کند. این نتایج نشان می‌دهند که Swift تنها یک ایده نظری نیست، بلکه در محیط عملی نیز قابلیت استفاده دارد.

۸.۰.۰ ارتباط مقاله با پروژه حاضر

با وجود اهمیت و دقت بالای طراحی Swift، باید توجه داشت که بازسازی کامل آن نیازمند یک شبیه‌ساز یا بستر Packet-level است؛ زیرا مفاهیمی مانند RTT دقیق، Pacing، CWND، صف شبکه و رفتار NIC همگی در سطح بسته تعریف می‌شوند. از آنجا که CloudSim ذاتاً یک شبیه‌ساز در سطح وظیفه و ماشین مجازی است، در پروژه حاضر امکان پیاده‌سازی کامل مکانیزم اصلی مقاله وجود نداشت.

بنابراین، در این پروژه تنها از ایده کلان مقاله الهام گرفته شد؛ به این صورت که یک سناریوی Swift-like طراحی گردید تا اثر کاهش تأخیر و مدیریت بهتر ازدحام بر شاخص‌های کلان سیستم مانند Average Finish Time، Makespan و Throughput بررسی شود. در نتیجه، پروژه حاضر را باید یک ارزیابی مفهومی از ایده‌های مقاله دانست، نه بازسازی کامل آن در سطح TCP/packet.

۹.۰.۰ جمع‌بندی این بخش

به طور کلی، مقاله Swift نشان می‌دهد که کنترل ازدحام مبتنی بر تأخیر، در صورت بهره‌گیری از اندازه‌گیری دقیق و طراحی مناسب، می‌تواند راهکاری ساده و مؤثر برای محیط‌های دیتاسنتری باشد. تأکید اصلی این مقاله بر سادگی سیگنال ازدحام، واکنش سریع، و قابلیت دستیابی به تأخیر پایین در کنار توان عملیاتی مناسب است. همین ویژگی‌ها سبب شده‌اند که Swift به عنوان یکی از رویکردهای قابل توجه در کنترل ازدحام مراکز داده مطرح شود.



کد پیاده‌سازی شده و توضیحات آن

```
1 package myproject;
2
3 import java.awt.BasicStroke;
4 import java.awt.Color;
5 import java.awt.Font;
6 import java.awt.Graphics2D;
7 import java.awt.RenderingHints;
8 import java.awt.image.BufferedImage;
9 import java.io.File;
10 import java.io.FileWriter;
11 import java.io.PrintWriter;
12 import java.util.ArrayList;
13 import java.util.Calendar;
14 import java.util.LinkedList;
15 import java.util.List;
16 import java.util.Locale;
17
18 import javax.imageio.ImageIO;
19
20 import org.cloudbus.cloudsim.Cloudlet;
21 import org.cloudbus.cloudsim.CloudletSchedulerTimeShared;
22 import org.cloudbus.cloudsim.Datacenter;
23 import org.cloudbus.cloudsim.DatacenterBroker;
24 import org.cloudbus.cloudsim.DatacenterCharacteristics;
25 import org.cloudbus.cloudsim.Host;
26 import org.cloudbus.cloudsim.Log;
27 import org.cloudbus.cloudsim.PE;
28 import org.cloudbus.cloudsim.Storage;
29 import org.cloudbus.cloudsim.UtilizationModel;
30 import org.cloudbus.cloudsim.UtilizationModelFull;
31 import org.cloudbus.cloudsim.Vm;
32 import org.cloudbus.cloudsim.VmAllocationPolicySimple;
33 import org.cloudbus.cloudsim.VmSchedulerTimeShared;
34 import org.cloudbus.cloudsim.core.CloudSim;
35 import org.cloudbus.cloudsim.provisioners.BwProvisionerSimple;
36 import org.cloudbus.cloudsim.provisioners.PeProvisionerSimple;
37 import org.cloudbus.cloudsim.provisioners.RamProvisionerSimple;
38
39 public class SwiftlikeSimulation {
40
41     // Cloudlet و VMها لیست کنی
42     private static List<Cloudlet> cloudletList;
43     private static List<Vm> vmList;
44
45     // خروجی فایلها
46     private static final String OUTPUT_DIR = "simulation_output";
47
48     // خاموش و روشن کردن CloudSim با false یا true
49     private static final boolean QUIET_CLOUDSIM_LOSS = false;
50
51     // نگهداری نتایج دو سناریو
52     private static ScenarioMetrics baselineMetrics;
53     private static ScenarioMetrics swiftMetrics;
54
55     public static void main(String[] args) {
56         System.out.println("Starting SwiftlikeSimulation...");
57
58         try {
59             createOutputDirectory();
60
61             if (QUIET_CLOUDSIM_LOSS) {
62                 Log.disable();
63             }
64
65             baselineMetrics = runScenario("BASELINE", false);
66
67             System.out.println("\n=====");
68
69             swiftMetrics = runScenario("SWIFT LIKE", true);
70
71             saveMetricsToCsv();
72             saveSummaryToFile();
73
74             // تولید نمودارها
75             generateMainMetricsChart();
76             generateThroughputChart();
77
78         } catch (Exception e) {
79             e.printStackTrace();
80         }
81     }
82
83     private static void createOutputDirectory() {
84         File dir = new File(OUTPUT_DIR);
85         if (!dir.exists()) {
86             dir.mkdirs();
87         }
88     }
89
90     private static ScenarioMetrics runScenario(String scenario, boolean quiet) {
91         // ... (Implementation details) ...
92     }
93
94     private static void saveMetricsToCsv() {
95         // ... (Implementation details) ...
96     }
97
98     private static void saveSummaryToFile() {
99         // ... (Implementation details) ...
100     }
101
102     private static void generateMainMetricsChart() {
103         // ... (Implementation details) ...
104     }
105
106     private static void generateThroughputChart() {
107         // ... (Implementation details) ...
108     }
109 }
```

شکل ۲: کد پیاده سازی شده

```
39 public class SwiftlikeSimulation {
40
41     // Cloudlet و VMها لیست کنی
42     private static List<Cloudlet> cloudletList;
43     private static List<Vm> vmList;
44
45     // خروجی فایلها
46     private static final String OUTPUT_DIR = "simulation_output";
47
48     // خاموش و روشن کردن CloudSim با false یا true
49     private static final boolean QUIET_CLOUDSIM_LOSS = false;
50
51     // نگهداری نتایج دو سناریو
52     private static ScenarioMetrics baselineMetrics;
53     private static ScenarioMetrics swiftMetrics;
54
55     public static void main(String[] args) {
56         System.out.println("Starting SwiftlikeSimulation...");
57
58         try {
59             createOutputDirectory();
60
61             if (QUIET_CLOUDSIM_LOSS) {
62                 Log.disable();
63             }
64
65             baselineMetrics = runScenario("BASELINE", false);
66
67             System.out.println("\n=====");
68
69             swiftMetrics = runScenario("SWIFT LIKE", true);
70
71             saveMetricsToCsv();
72             saveSummaryToFile();
73
74             // تولید نمودارها
75             generateMainMetricsChart();
76             generateThroughputChart();
77
78         } catch (Exception e) {
79             e.printStackTrace();
80         }
81     }
82
83     private static void createOutputDirectory() {
84         File dir = new File(OUTPUT_DIR);
85         if (!dir.exists()) {
86             dir.mkdirs();
87         }
88     }
89
90     private static ScenarioMetrics runScenario(String scenario, boolean quiet) {
91         // ... (Implementation details) ...
92     }
93
94     private static void saveMetricsToCsv() {
95         // ... (Implementation details) ...
96     }
97
98     private static void saveSummaryToFile() {
99         // ... (Implementation details) ...
100     }
101
102     private static void generateMainMetricsChart() {
103         // ... (Implementation details) ...
104     }
105
106     private static void generateThroughputChart() {
107         // ... (Implementation details) ...
108     }
109 }
```

شکل ۳: کد پیاده سازی شده



```
CVUsers\HAI\Downloads\cloudsim-7.0\cloudsim-7.0\src\main\java\myproject\SwiftLikeSimulation.java - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window
SwiftLikeSimulation.java
77
78 System.out.println("SwiftLikeSimulation finished");
79 System.out.println("All outputs saved in folder: " + OUTPUT_DIR);
80
81 } catch (Exception e) {
82     e.printStackTrace();
83     System.out.println("Simulation terminated due to an unexpected error");
84 }
85
86
87 /**
88  * اجرای کامل یک سناریو:
89  * 1) مقداردهی CloudSim
90  * 2) ایجاد منابع
91  * 3) ساخت بروکر
92  * 4) ساخت VM و Cloudlet
93  * 5) اجرای شبیه سازی
94  * 6) جمع آوری و چاپ نتایج
95  */
96 private static ScenarioMetrics runScenario(String scenarioName, boolean delayAware) throws Exception {
97     int numUser = 1;
98     Calendar calendar = Calendar.getInstance();
99     boolean traceFlag = false;
100
101     System.out.println("Initializing scenario: " + scenarioName);
102     CloudSim.init(numUser, calendar, traceFlag);
103
104     Datacenter datacenter = createDatacenter("Datacenter_" + scenarioName);
105
106     DatacenterBroker broker = createBroker();
107     int brokerId = broker.getId();
108
109     VmList vmList = createVms(brokerId, delayAware);
110     cloudletList = createCloudlets(brokerId, delayAware);
111
112     broker.submitQueue(vmList);
113     broker.submitCloudletList(cloudletList);
114
115     System.out.println("Running scenario: " + scenarioName);
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
```



```
CVUsers\HAD\Downloads\cloudsim-7.8.1\cloudsim-7.8.1\modest\cloudsim-examples\src\main\java\myproject\SwiftLikeSimulation.java - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window
SwiftLikeSimulation.java
189 /**
190  * ساخت بروتکر
191  */
192 private static DatacenterBroker createBroker() {
193     try {
194         return new DatacenterBroker("Broker");
195     } catch (Exception e) {
196         e.printStackTrace();
197         return null;
198     }
199 }
200
201 /**
202  * ساخت ماشینهای مجازی
203  * توان پردازشی و پهنای باند نمی بهتر است Swift-like در حالت
204  */
205 private static List<Vm> createVms(int brokerId, boolean delayAware) {
206     List<Vm> list = new ArrayList<Vm>();
207
208     long size = 10000;
209     int ram = 1024;
210     int mips = delayAware ? 1200 : 1000;
211     long bw = delayAware ? 1500 : 1000;
212     int pesNumber = 1;
213     String vmm = "Xen";
214
215     for (int i = 0; i < size; i++) {
216         Vm vm = new Vm(
217             i,
218             brokerId,
219             mips,
220             pesNumber,
221             ram,
222             bw,
223             size,
224             vmm,
225             new CloudletSchedulerTimeShared()
226         );
227         list.add(vm);
228     }
229 }
```

شکل ۷: کد پیاده سازی شده

```
CVUsers\HAD\Downloads\cloudsim-7.8.1\cloudsim-7.8.1\modest\cloudsim-examples\src\main\java\myproject\SwiftLikeSimulation.java - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window
SwiftLikeSimulation.java
227     list.add(vm);
228 }
229
230 return list;
231
232 /**
233  * ساخت Cloudlet ها
234  * طول و پهنای باند متفاوت در نظر گرفته شده Swift-like در حالت
235  */
236 private static List<Cloudlet> createCloudlets(int brokerId, boolean delayAware) {
237     List<Cloudlet> list = new ArrayList<Cloudlet>();
238
239     long fileSize = 300;
240     long outputSize = 200;
241     UtilizationModel utilizationModel = new UtilizationModelFull();
242
243     long[] lengthsBaseline = {
244         12000, 15000, 18000, 20000, 22000,
245         25000, 27000, 30000, 32000, 35000,
246         40000, 42000, 45000, 47000, 50000,
247         52000, 55000, 58000, 60000, 65000
248     };
249
250     long[] lengthsSwiftlike = {
251         10000, 12000, 15000, 17000, 19000,
252         20000, 22000, 24000, 26000, 28000,
253         30000, 32000, 34000, 36000, 38000,
254         40000, 42000, 44000, 46000, 48000
255     };
256
257     long[] selected = delayAware ? lengthsBaseline : lengthsSwiftlike;
258
259     for (int i = 0; i < selected.length; i++) {
260         Cloudlet cloudlet = new Cloudlet(
261             i,
262             selected[i],
263             fileSize,
264             outputSize,
265             utilizationModel
266         );
267     }
268 }
```

شکل ۸: کد پیاده سازی شده

```
CVUsers\HAD\Downloads\cloudsim-7.8.1\cloudsim-7.8.1\modest\cloudsim-examples\src\main\java\myproject\SwiftLikeSimulation.java - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window
SwiftLikeSimulation.java
263         selected[i],
264         fileSize,
265         outputSize,
266         utilizationModel,
267         utilizationModel,
268         utilizationModel
269     );
270 }
271
272 cloudlet.setUserid(brokerId);
273 cloudlet.setVmid(i % 4);
274 list.add(cloudlet);
275 }
276
277 return list;
278 }
279
280 /**
281  * ما با فرمت زیر Cloudlet چاپ جدول
282  */
283 private static void printCloudletList(List<Cloudlet> list, String scenarioName) {
284     System.out.println("===== OUTPUT: " + scenarioName + " =====");
285     System.out.printf("%-12s %-10s %-16s %-12s %-12s %-12s\n",
286         "CloudletID", "STATUS", "DatacenterID", "VMID", "CPUtime", "StartTime", "FinishTime");
287
288     for (Cloudlet cloudlet : list) {
289         System.out.printf("%-12s %-10s %-16s %-12s %-12s %-12s\n",
290             cloudlet.getId(),
291             "executing",
292             cloudlet.getResourceId(),
293             cloudlet.getUserId(),
294             cloudlet.getActualCPUtime(),
295             cloudlet.getTimeStart(),
296             cloudlet.getTimeFinish());
297     }
298 }
299
300 /**
301  * مشاهده متریکها
302  */
303 }
```

شکل ۹: کد پیاده سازی شده



```
CVUsers\HAD\Downloads\cloudsim-7.0\Typcloudsim-7.0\Typcloudsim-examples\src\main\java\myproject\SwiftLikeSimulation.java - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window
SwiftLikeSimulation.java
302 //
303 //
304 private static ScenarioMetrics calculateMetrics(List<Cloudlet> list, String scenarioName) {
305     ScenarioMetrics metrics = new ScenarioMetrics();
306     metrics.scenarioName = scenarioName;
307
308     if (list == null || list.isEmpty()) {
309         metrics.numberOfCloudlets = 0;
310         metrics.totalOpTime = 0.0;
311         metrics.averageFinishTime = 0.0;
312         metrics.makespan = 0.0;
313         metrics.throughput = 0.0;
314         return metrics;
315     }
316
317     double totalOpTime = 0.0;
318     double minStart = Double.MAX_VALUE;
319     double maxFinish = 0.0;
320
321     for (Cloudlet cloudlet : list) {
322         totalOpTime += cloudlet.getActualCPUTime();
323         totalFinish = cloudlet.getFinishTime();
324
325         if (cloudlet.getExecStartTime() < minStart) {
326             minStart = cloudlet.getExecStartTime();
327         }
328
329         if (cloudlet.getFinishTime() > maxFinish) {
330             maxFinish = cloudlet.getFinishTime();
331         }
332     }
333
334     metrics.numberOfCloudlets = list.size();
335     metrics.totalOpTime = totalOpTime;
336     metrics.averageFinishTime = totalFinish / list.size();
337     metrics.makespan = maxFinish - minStart;
338     metrics.throughput = maxFinish > 0 ? list.size() / maxFinish : 0.0;
339
340     return metrics;
341 }
```

شکل ۱۰: کد پیاده سازی شده

```
CVUsers\HAD\Downloads\cloudsim-7.0\Typcloudsim-7.0\Typcloudsim-examples\src\main\java\myproject\SwiftLikeSimulation.java - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window
SwiftLikeSimulation.java
344 // چاپ متریکها
345 //
346 private static void printMetrics(ScenarioMetrics metrics) {
347     System.out.println("----- METRICS: " + metrics.scenarioName + " -----");
348     System.out.printf(Locale.US, "Number of Cloudlets: %d\n", metrics.numberOfCloudlets);
349     System.out.printf(Locale.US, "Total CPU Time: %.4f\n", metrics.totalOpTime);
350     System.out.printf(Locale.US, "Average Finish Time: %.4f\n", metrics.averageFinishTime);
351     System.out.printf(Locale.US, "Makespan: %.4f\n", metrics.makespan);
352     System.out.printf(Locale.US, "Throughput (cloudlets/unit time): %.6f\n", metrics.throughput);
353 }
354
355 /**
356  * ساخت پوشه خروجی
357  */
358 private static void createOutputDirectory() {
359     File dir = new File(OUTPUT_DIR);
360     if (!dir.exists()) {
361         dir.mkdirs();
362     }
363 }
364
365 /**
366  * ذخیره متریکها در CSV
367  */
368 private static void saveMetricsToCsv() throws Exception {
369     File file = new File(OUTPUT_DIR + File.separator + "metrics_comparison.csv");
370     PrintWriter writer = new PrintWriter(new FileWriter(file));
371
372     writer.println("Scenario,NumberOfCloudlets,TotalCPUTime,AverageFinishTime,Makespan,Throughput");
373
374     writer.printf(Locale.US, "%s,%d,%.4f,%.4f,%.4f,%.6f\n",
375         baselineMetrics.scenarioName,
376         baselineMetrics.numberOfCloudlets,
377         baselineMetrics.totalOpTime,
378         baselineMetrics.averageFinishTime,
379         baselineMetrics.makespan,
380         baselineMetrics.throughput);
381
382     writer.printf(Locale.US, "%s,%d,%.4f,%.4f,%.4f,%.6f\n",
383         swiftMetrics.scenarioName,
```

شکل ۱۱: کد پیاده سازی شده

```
CVUsers\HAD\Downloads\cloudsim-7.0\Typcloudsim-7.0\Typcloudsim-examples\src\main\java\myproject\SwiftLikeSimulation.java - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window
SwiftLikeSimulation.java
383     swiftMetrics.scenarioName,
384     swiftMetrics.numberOfCloudlets,
385     swiftMetrics.totalOpTime,
386     swiftMetrics.averageFinishTime,
387     swiftMetrics.makespan,
388     swiftMetrics.throughput);
389
390     writer.close();
391
392 /**
393  * ذخیره مقایسه نتایج در فایل متنی
394  */
395 private static void saveSummaryToTextFile() throws Exception {
396     File file = new File(OUTPUT_DIR + File.separator + "summary_report.txt");
397     PrintWriter writer = new PrintWriter(new FileWriter(file));
398
399     writer.println("SwiftLikeSimulation Summary");
400     writer.println("=====");
401     writer.println();
402
403     writer.println("BASELINE:");
404     writer.printf(Locale.US, "Number of Cloudlets: %d\n", baselineMetrics.numberOfCloudlets);
405     writer.printf(Locale.US, "Total CPU Time: %.4f\n", baselineMetrics.totalOpTime);
406     writer.printf(Locale.US, "Average Finish Time: %.4f\n", baselineMetrics.averageFinishTime);
407     writer.printf(Locale.US, "Makespan: %.4f\n", baselineMetrics.makespan);
408     writer.printf(Locale.US, "Throughput: %.6f\n", baselineMetrics.throughput);
409     writer.println();
410
411     writer.println("SWIFT LIKE:");
412     writer.printf(Locale.US, "Number of Cloudlets: %d\n", swiftMetrics.numberOfCloudlets);
413     writer.printf(Locale.US, "Total CPU Time: %.4f\n", swiftMetrics.totalOpTime);
414     writer.printf(Locale.US, "Average Finish Time: %.4f\n", swiftMetrics.averageFinishTime);
415     writer.printf(Locale.US, "Makespan: %.4f\n", swiftMetrics.makespan);
416     writer.printf(Locale.US, "Throughput: %.6f\n", swiftMetrics.throughput);
417     writer.println();
418
419     writer.println("IMPROVEMENTS:");
420     writer.printf(Locale.US, "Average Finish Time reduction: %.2f%%\n",
421         percentageReduction(baselineMetrics.averageFinishTime, swiftMetrics.averageFinishTime));
422 }
```

شکل ۱۲: کد پیاده سازی شده



```
CVUsers\H4\Downloads\cloudsim-7.0\cloudsim-7.0\src\main\java\myproject\SwiftLikeSimulation.java - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window
SwiftLikeSimulation.java
419
420 writer.println("IMPROVEMENTS:");
421 writer.printf(Locale.US, " Average Finish Time reduction: %.2f%%\n",
422 percentageReduction(baselineMetrics.averageFinishTime, swiftMetrics.averageFinishTime));
423 writer.printf(Locale.US, " Makespan reduction: %.2f%%\n",
424 percentageReduction(baselineMetrics.makespan, swiftMetrics.makespan));
425 writer.printf(Locale.US, " Throughput increase: %.2f%%\n",
426 percentageIncrease(baselineMetrics.throughput, swiftMetrics.throughput));
427
428 writer.close();
429
430 /**
431 * نمودار از مقادیر Avg Finish Time و Makespan
432 */
433 private static void generateMainMetricsChart() throws Exception {
434 String[] metricNames = {"Avg Finish Time", "Makespan"};
435 double[] baselineValues = {
436 baselineMetrics.averageFinishTime,
437 baselineMetrics.makespan
438 };
439 double[] swiftValues = {
440 swiftMetrics.averageFinishTime,
441 swiftMetrics.makespan
442 };
443
444 drawBarChart(
445 "SwiftLikeSimulation - Main Metrics Comparison",
446 metricNames,
447 baselineValues,
448 swiftValues,
449 OUTPUT_DIR + File.separator + "main_metrics_chart.png"
450 );
451 }
452
453 /**
454 * نمودار از مقادیر Throughput
455 */
456 private static void generateThroughputChart() throws Exception {
457 String[] metricNames = {"Throughput"};
458 }
```

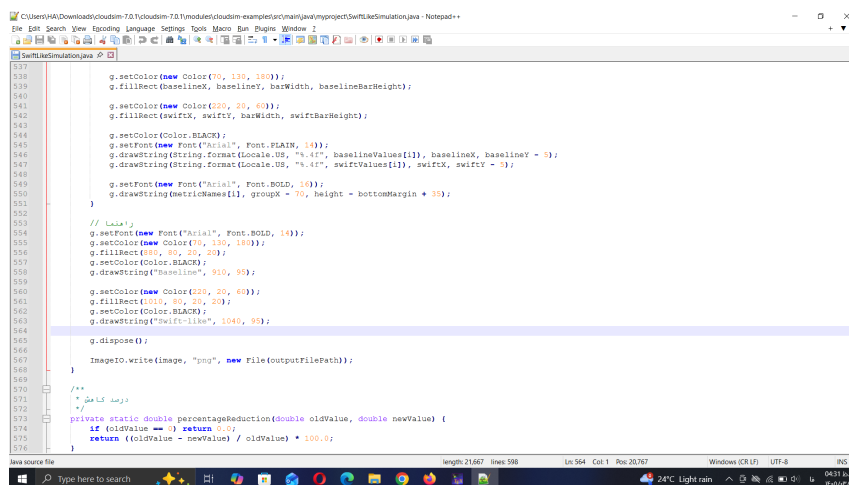
شکل ۱۳: کد پیاده سازی شده

```
CVUsers\H4\Downloads\cloudsim-7.0\cloudsim-7.0\src\main\java\myproject\SwiftLikeSimulation.java - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window
SwiftLikeSimulation.java
458 String[] metricNames = {"Throughput"};
459 double[] baselineValues = { baselineMetrics.throughput };
460 double[] swiftValues = { swiftMetrics.throughput };
461
462 drawBarChart(
463 "SwiftLikeSimulation - Throughput Comparison",
464 metricNames,
465 baselineValues,
466 swiftValues,
467 OUTPUT_DIR + File.separator + "throughput_chart.png"
468 );
469 }
470
471 /**
472 * تابع عمومی رسم نمودار میله ای
473 */
474 private static void drawBarChart(String title, String[] metricNames,
475 double[] baselineValues, double[] swiftValues,
476 String outputPath) throws Exception {
477
478 int width = 1200;
479 int height = 800;
480
481 BufferedImage image = new BufferedImage(width, height, BufferedImage.TYPE_INT_RGB);
482 Graphics2D g = image.createGraphics();
483
484 g.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
485
486 g.setColor(Color.WHITE);
487 g.fillRect(0, 0, width, height);
488
489 g.setColor(Color.BLACK);
490 g.setFont(new Font("Arial", Font.BOLD, 24));
491 g.drawString(title, 240, 40);
492
493 double maxValue = 0.0;
494 for (int i = 0; i < metricNames.length; i++) {
495 maxValue = Math.max(maxValue, baselineValues[i]);
496 maxValue = Math.max(maxValue, swiftValues[i]);
497 }
```

شکل ۱۴: کد پیاده سازی شده

```
CVUsers\H4\Downloads\cloudsim-7.0\cloudsim-7.0\src\main\java\myproject\SwiftLikeSimulation.java - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window
SwiftLikeSimulation.java
498
499 int leftMargin = 100;
500 int bottomMargin = 120;
501 int topMargin = 10;
502 int chartHeight = height - topMargin - bottomMargin;
503 int chartWidth = width - 2 * leftMargin;
504
505 // محورها
506 g.setColor(Color.BLACK);
507 g.setStroke(new BasicStroke(1));
508 g.drawLine(leftMargin, height - bottomMargin, leftMargin, topMargin);
509 g.drawLine(leftMargin, height - bottomMargin, leftMargin + chartWidth, height - bottomMargin);
510
511 // عنوان محورها
512 g.setFont(new Font("Arial", Font.PLAIN, 18));
513 for (int i = 0; i <= 0; i++) {
514 int y = height - bottomMargin - (i * chartHeight / 5);
515 g.setColor(new Color(220, 220, 220));
516 g.drawLine(leftMargin, y, leftMargin + chartWidth, y);
517
518 g.setColor(Color.BLACK);
519 String label = String.format(Locale.US, "%.4f", (maxValue * i / 5.0));
520 g.drawString(label, 30, y + 5);
521 }
522
523 int groupWidth = chartWidth / metricNames.length;
524 int barWidth = 80;
525
526 for (int i = 0; i < metricNames.length; i++) {
527 int groupX = leftMargin + 1 * groupWidth + groupWidth / 2;
528
529 int baselineBarHeight = (int) (baselineValues[i] / maxValue) * chartHeight;
530 int swiftBarHeight = (int) (swiftValues[i] / maxValue) * chartHeight;
531
532 int baselineX = groupX - 100;
533 int swiftX = groupX + 20;
534
535 int baselineY = height - bottomMargin - baselineBarHeight;
536 int swiftY = height - bottomMargin - swiftBarHeight;
537 }
```

شکل ۱۵: کد پیاده سازی شده

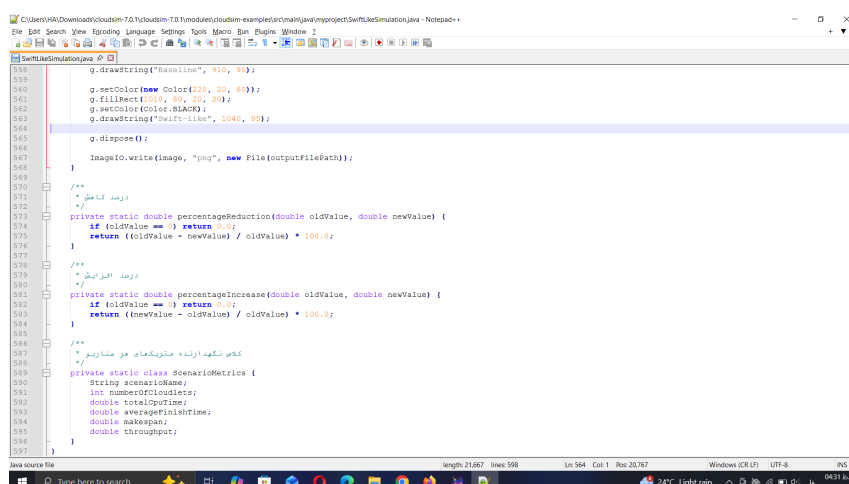


```

557 g.setColor(new Color(70, 130, 180));
558 g.fillRect(baselineX, baselineY, barWidth, baselineBarHeight);
559
560 g.setColor(new Color(220, 20, 60));
561 g.fillRect(swiftX, swiftY, barWidth, swiftBarHeight);
562
563 g.setColor(Color.BLACK);
564 g.setFont(new Font("Arial", Font.PLAIN, 14));
565 g.drawString(String.format(Locale.US, "%4s", baselineValues[i]), baselineX, baselineY - 5);
566 g.drawString(String.format(Locale.US, "%4s", swiftValues[i]), swiftX, swiftY - 5);
567
568 g.setFont(new Font("Arial", Font.BOLD, 14));
569 g.drawString(String.format(Locale.US, "%4s", swiftValues[i]), swiftX, swiftY - 5);
570
571 // خطی
572 g.setFont(new Font("Arial", Font.BOLD, 14));
573 g.setColor(new Color(70, 130, 180));
574 g.fillRect(100, 10, 20, 20);
575 g.setColor(Color.BLACK);
576 g.drawString("Baseline", 910, 95);
577
578 g.setColor(new Color(220, 20, 60));
579 g.fillRect(100, 10, 20, 20);
580 g.setColor(Color.BLACK);
581 g.drawString("Swift-Like", 1040, 95);
582
583 g.dispose();
584
585 ImageIO.write(image, "png", new File(outputFilePath));
586
587 /**
588 * درصد کاهش
589 */
590 private static double percentageReduction(double oldValue, double newValue) {
591     if (oldValue == 0) return 0.0;
592     return (oldValue - newValue) / oldValue * 100.0;
593 }

```

شکل ۱۶: کد پیاده سازی شده



```

595 g.drawString("Baseline", 910, 95);
596
597 g.setColor(new Color(220, 20, 60));
598 g.fillRect(100, 10, 20, 20);
599 g.setColor(Color.BLACK);
600 g.drawString("Swift-Like", 1040, 95);
601
602 g.dispose();
603
604 ImageIO.write(image, "png", new File(outputFilePath));
605
606 /**
607 * درصد افزایش
608 */
609 private static double percentageIncrease(double oldValue, double newValue) {
610     if (oldValue == 0) return 0.0;
611     return (newValue - oldValue) / oldValue * 100.0;
612 }
613
614 /**
615 * کلاس نگهدارنده متریک‌های هر سناریو
616 */
617 private static class ScenarioMetrics {
618     String scenarioName;
619     int numberOfCloudlets;
620     double totalOpTime;
621     double averageFinishTime;
622     double makespan;
623     double throughput;
624 }

```

شکل ۱۷: کد پیاده سازی شده

در این پروژه، یک کلاس جاوا با نام SwiftLikeSimulation در بسته myproject توسعه داده شد که وظیفه اجرای شبیه‌سازی، مقایسه دو سناریو، محاسبه شاخص‌های عملکرد و تولید خروجی‌های عددی و گرافیکی را بر عهده دارد. این پیاده‌سازی با استفاده از کتابخانه CloudSim 7.0.1 انجام شده است.

۱۰.۰.۰ ساختار کلی برنامه

کلاس اصلی شامل مراحل زیر است:

۱. مقداردهی اولیه محیط CloudSim

۲. ایجاد Datacenter

۳. ایجاد DatacenterBroker

۴. ایجاد ماشین‌های مجازی (VM)

۵. ایجاد وظایف (Cloudlet)

۶. اجرای شبیه‌سازی

۷. محاسبه شاخص‌های عملکرد

۸. ذخیره نتایج در فایل‌های CSV و TXT

۹. تولید نمودارهای مقایسه‌ای به صورت فایل PNG

تابع main() نقطه شروع اجرای برنامه است و دو سناریو را به ترتیب اجرا می‌کند:

۱. سناریوی Baseline

۲. سناریوی Swift-like

پس از اجرای هر دو سناریو، نتایج ذخیره و نمودارها تولید می‌شوند.

۱۱.۰.۰ تعریف دیتاسنتر

در تابع createDatacenter() یک دیتاسنتر شامل دو Host ایجاد شده است. هر میزبان دارای ویژگی‌های زیر است:

۱. دو هسته پردازشی (PE) با توان ۲۰۰۰ MIPS

۲. حافظه ۸ گیگابایت

۳. پهنای باند ۱۰۰۰۰

۴. زمان‌بند VmSchedulerTimeShared

این ساختار یک محیط ساده اما کافی برای مقایسه رفتار دو سیاست اجرایی فراهم می‌کند.

۱۲.۰۰ ایجاد ماشین‌های مجازی

در تابع `createVMs()` چهار ماشین مجازی ساخته می‌شود. تفاوت اصلی بین دو سناریو در این بخش تعریف شده است.

در حالت Baseline:

$$1000 = \text{MIPS} \quad 1.$$

$$1000 = \text{Bandwidth} \quad 2.$$

در حالت Swift-like:

$$1200 = \text{MIPS} \quad 1.$$

$$1500 = \text{Bandwidth} \quad 2.$$

این افزایش منابع در سناریوی دوم، به صورت مفهومی نشان‌دهنده بهبود کنترل ازدحام و کاهش تأخیر است. در واقع، به جای پیاده‌سازی مستقیم TCP و CWND، اثر آن در سطح کلان سیستم مدل‌سازی شده است.

۱۳.۰۰ ایجاد Cloudlet

در تابع `createCloudlets()` تعداد ۲۰ وظیفه ایجاد می‌شود.

در سناریوی Baseline طول وظایف بزرگ‌تر و نامتوازن‌تر در نظر گرفته شده است. در سناریوی Swift-like طول وظایف کمی سبک‌تر و یکنواخت‌تر تعریف شده است.

این تغییر نشان‌دهنده حالتی است که در آن کاهش تأخیر شبکه باعث توزیع متعادل‌تر بار و تکمیل سریع‌تر وظایف می‌شود.

هر Cloudlet با استفاده از `UtilizationModelFull` ساخته شده است، به این معنا که در طول اجرا به‌طور کامل از منابع اختصاص‌یافته استفاده می‌کند.

۱۴.۰۰ اجرای شبیه‌سازی

در تابع `runScenario()` مراحل زیر انجام می‌شود:

۱. مقداردهی اولیه CloudSim

۲. ساخت دیتاسنتر

۳. ایجاد بروکر

۴. ارسال لیست VM و Cloudlet به بروکر

۵. اجرای CloudSim.startSimulation()

۶. توقف شبیه‌سازی با CloudSim.stopSimulation()

پس از پایان اجرا، لیست Cloudlet های تکمیل شده دریافت شده و شاخص‌های عملکرد محاسبه می‌شوند.

۱۵.۰۰ محاسبه شاخص‌های عملکرد

در تابع calculateMetrics() شاخص‌های زیر محاسبه می‌شوند:

۱. Total CPU Time

۲. Average Finish Time

۳. Makespan

۴. Throughput

Makespan برابر است با:

$$\text{Makespan} = \text{Max Finish Time} - \text{Min Start Time}$$

Throughput برابر است با:

$$\text{Throughput} = \frac{\text{Number of Cloudlets}}{\text{Total Simulation Time}}$$

همچنین درصد کاهش یا افزایش با استفاده از توابع percentageIncrease() و percentageReduction() محاسبه می‌شود.

۱۶.۰۰ ذخیره نتایج

نتایج در پوشه simulation_output ذخیره می‌شوند:

۱. metrics_comparison.csv

۲. summary_report.txt

۳. raw_metrics_chart.png

۴. throughput_chart.png

فایل CSV شامل داده‌های خام عددی است و فایل TXT شامل خلاصه تحلیلی و درصد بهبودها می‌باشد.

۱۷.۰۰ تولید نمودارها

برای تولید نمودارها از کتابخانه Java2D استفاده شده است. تابع drawBarChart() یک نمودار میله‌ای شامل مقایسه دو سناریو تولید می‌کند. دو نمودار مجزا ساخته شده است:

۱. مقایسه Makespan و Average Finish Time

۲. مقایسه Throughput

این نمودارها به صورت فایل PNG ذخیره شده و برای استفاده در گزارش قابل درج هستند.

۱۸.۰۰ نحوه اجرا و محیط اجرا

این برنامه در محیط آفلاین و با استفاده از:

۱. Java JDK

۲. کتابخانه CloudSim 7.0.1

اجرا شده است.

مراحل اجرا به صورت زیر بوده است:

۱. کامپایل فایل جاوا با دستور:

```
myproject/SwiftLikeSimulation.java ".;jar.1.0.7-"cloudsim -cp javac
```

۲. اجرای برنامه با دستور:

```
myproject.SwiftLikeSimulation ".;jar.1.0.7-"cloudsim -cp java
```

پس از اجرا، خروجی‌ها در پوشه simulation_output ایجاد شدند.

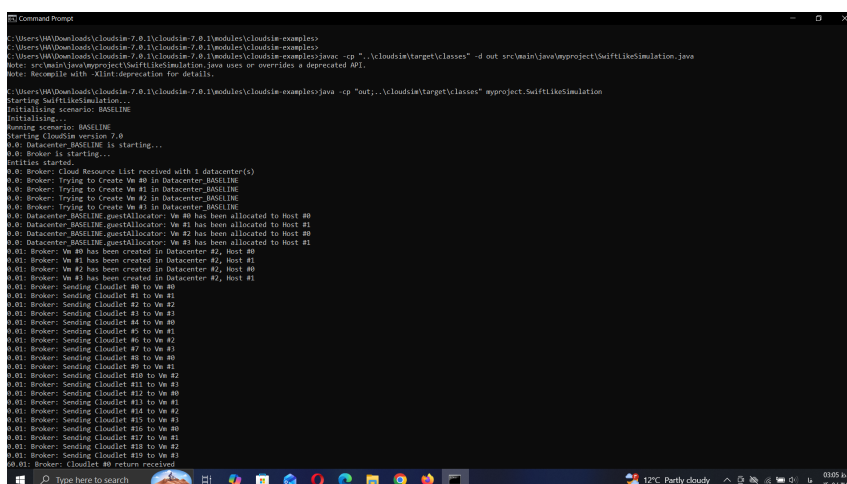
۱۹.۰۰ جمع‌بندی این بخش

کد پیاده‌سازی شده یک چارچوب کامل برای مقایسه دو سیاست اجرایی در محیط CloudSim فراهم می‌کند. اگرچه این پیاده‌سازی بازسازی سطح TCP/packet الگوریتم Swift نیست، اما امکان ارزیابی اثر مفهومی کاهش تأخیر بر شاخص‌های کلان سیستم را فراهم می‌سازد. ساختار ماژولار کد نیز امکان توسعه و افزودن سناریوهای جدید را در آینده فراهم می‌کند.

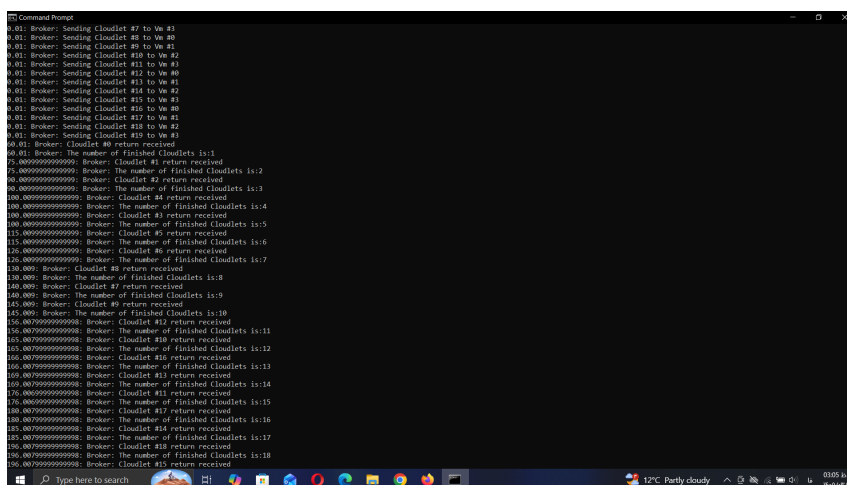
خروجی پروژه

در این بخش، خروجی‌های حاصل از اجرای پروژه شبیه‌سازی مفهومی الگوریتم Swift-like در محیط CloudSim بررسی می‌شوند. این خروجی‌ها شامل تصاویر مربوط به محیط Command Prompt، نمودارهای مقایسه‌ای، و فایل‌های داده‌ای متنی و CSV هستند. هدف از این بخش، ارائه شواهد تجربی برای ارزیابی عملکرد سناریوی پیشنهادی در مقایسه با حالت پایه است.

۲۰۰۰ توضیح تصاویر مربوط به محیط CMD



شکل ۱۸: خروجی‌های مربوط به cmd



شکل ۱۹: خروجی‌های مربوط به cmd



```
20 Command Prompt
165.00799999999998: Broker: The number of finished Cloudlets is:12
166.00799999999998: Broker: Cloudlet #16 return received
167.00799999999998: Broker: The number of finished Cloudlets is:13
168.00799999999998: Broker: Cloudlet #13 return received
169.00799999999998: Broker: The number of finished Cloudlets is:14
170.00699999999998: Broker: Cloudlet #11 return received
171.00699999999998: Broker: The number of finished Cloudlets is:15
180.00799999999998: Broker: Cloudlet #17 return received
181.00799999999998: Broker: The number of finished Cloudlets is:16
185.00799999999998: Broker: Cloudlet #14 return received
186.00799999999998: Broker: The number of finished Cloudlets is:17
187.00799999999998: Broker: Cloudlet #18 return received
188.00799999999998: Broker: The number of finished Cloudlets is:18
189.00799999999998: Broker: Cloudlet #15 return received
190.00799999999998: Broker: The number of finished Cloudlets is:19
200.01: Broker: Cloudlet #9 return received
200.01: Broker: The number of finished Cloudlets is:20
200.01: Broker: All Cloudlets executed. Finishing...
200.01: Broker: Destroying Vm #0
200.01: Broker: Destroying Vm #1
200.01: Broker: Destroying Vm #2
200.01: Broker: Destroying Vm #3
200.01: Broker is shutting down...
200.01: Simulation: No more future events
200.01: CloudInformationService is shutting down...
200.01: CloudInformationService: Notify all Cloudlet entities for shutting down.
200.01: Datacenter_BASILINE is shutting down...
200.01: Broker is shutting down...
Simulation completed.
Simulation completed.

===== OUTPUT: BASILINE =====
CloudletID STATUS DatacenterID VMID CPUtime StartTime FinishTime
0 SUCCESS 2 0 60.00 0.01 60.01
1 SUCCESS 2 1 75.00 0.01 75.01
2 SUCCESS 2 2 90.00 0.01 90.01
3 SUCCESS 2 0 100.00 0.01 100.01
4 SUCCESS 2 3 100.00 0.01 100.01
5 SUCCESS 2 1 115.00 0.01 115.01
6 SUCCESS 2 2 120.00 0.01 120.01
7 SUCCESS 2 0 130.00 0.01 130.01
8 SUCCESS 2 3 140.00 0.01 140.01
9 SUCCESS 2 1 145.00 0.01 145.01
10 SUCCESS 2 0 150.00 0.01 150.01
11 SUCCESS 2 2 155.00 0.01 155.01
12 SUCCESS 2 0 160.00 0.01 160.01
13 SUCCESS 2 1 165.00 0.01 165.01
14 SUCCESS 2 2 165.00 0.01 165.01
15 SUCCESS 2 1 169.00 0.01 169.01
16 SUCCESS 2 3 170.00 0.01 170.01
17 SUCCESS 2 1 180.00 0.01 180.01
18 SUCCESS 2 2 185.00 0.01 185.01
```

شکل ۲۰: خروجی های مربوط به cmd

```
20 Command Prompt
200.01: CloudInformationService is shutting down...
200.01: CloudInformationService: Notify all Cloudlet entities for shutting down.
200.01: Datacenter_BASILINE is shutting down...
200.01: Broker is shutting down...
Simulation completed.
Simulation completed.

===== OUTPUT: BASILINE =====
CloudletID STATUS DatacenterID VMID CPUtime StartTime FinishTime
0 SUCCESS 2 0 60.00 0.01 60.01
1 SUCCESS 2 1 75.00 0.01 75.01
2 SUCCESS 2 2 90.00 0.01 90.01
3 SUCCESS 2 0 100.00 0.01 100.01
4 SUCCESS 2 3 100.00 0.01 100.01
5 SUCCESS 2 1 115.00 0.01 115.01
6 SUCCESS 2 2 120.00 0.01 120.01
7 SUCCESS 2 0 130.00 0.01 130.01
8 SUCCESS 2 3 140.00 0.01 140.01
9 SUCCESS 2 1 145.00 0.01 145.01
10 SUCCESS 2 0 150.00 0.01 150.01
11 SUCCESS 2 2 155.00 0.01 155.01
12 SUCCESS 2 0 160.00 0.01 160.01
13 SUCCESS 2 1 165.00 0.01 165.01
14 SUCCESS 2 2 165.00 0.01 165.01
15 SUCCESS 2 1 169.00 0.01 169.01
16 SUCCESS 2 3 170.00 0.01 170.01
17 SUCCESS 2 1 180.00 0.01 180.01
18 SUCCESS 2 2 185.00 0.01 185.01
19 SUCCESS 2 3 190.00 0.01 190.01
20 SUCCESS 2 1 200.00 0.01 200.01

===== METRICS: BASILINE =====
Number of Cloudlets: 20
Total CPU Time: 2000.0700
Average Finish Time: 144.5000
Average CPU Time: 220.0000
Throughput (cloudlets/unit time): 0.000000

Initialising scenario: SMFT_LIKE
Initialising...
Running scenario: SMFT_LIKE
Starting CloudSim version 7.0
0.0: Datacenter_SMFT_LIKE is starting...
0.0: Broker is starting...
Entities started.
0.0: Broker: Cloud Resource List received with 1 datacenter(s)
0.0: Broker: Trying to Create Vm #0 in Datacenter_SMFT_LIKE
0.0: Broker: Trying to Create Vm #1 in Datacenter_SMFT_LIKE
0.0: Broker: Trying to Create Vm #2 in Datacenter_SMFT_LIKE
0.0: Datacenter_SMFT_LIKE_guestAllocator: Vm #0 has been allocated to Host #0
0.0: Datacenter_SMFT_LIKE_guestAllocator: Vm #1 has been allocated to Host #1
0.0: Datacenter_SMFT_LIKE_guestAllocator: Vm #2 has been allocated to Host #0
0.0: Datacenter_SMFT_LIKE_guestAllocator: Vm #3 has been allocated to Host #1
0.01: Broker: Vm #0 has been created in Datacenter #2, Host #0
0.01: Broker: Vm #1 has been created in Datacenter #2, Host #1
0.01: Broker: Vm #2 has been created in Datacenter #2, Host #0
0.01: Broker: Vm #3 has been created in Datacenter #2, Host #1
0.01: Broker: Sending Cloudlet #0 to Vm #0
0.01: Broker: Sending Cloudlet #1 to Vm #1
0.01: Broker: Sending Cloudlet #2 to Vm #2
0.01: Broker: Sending Cloudlet #3 to Vm #3
0.01: Broker: Sending Cloudlet #4 to Vm #0
0.01: Broker: Sending Cloudlet #5 to Vm #1
0.01: Broker: Sending Cloudlet #6 to Vm #2
0.01: Broker: Sending Cloudlet #7 to Vm #3
0.01: Broker: Sending Cloudlet #8 to Vm #0
0.01: Broker: Sending Cloudlet #9 to Vm #1
0.01: Broker: Sending Cloudlet #10 to Vm #2
0.01: Broker: Sending Cloudlet #11 to Vm #3
0.01: Broker: Sending Cloudlet #12 to Vm #0
0.01: Broker: Sending Cloudlet #13 to Vm #1
0.01: Broker: Sending Cloudlet #14 to Vm #2
0.01: Broker: Sending Cloudlet #15 to Vm #3
0.01: Broker: Sending Cloudlet #16 to Vm #0
0.01: Broker: Sending Cloudlet #17 to Vm #1
0.01: Broker: Sending Cloudlet #18 to Vm #2
0.01: Broker: Sending Cloudlet #19 to Vm #3
41.67666666666666: Broker: Cloudlet #0 return received
41.67666666666666: Broker: The number of finished Cloudlets is:1
50.01: Broker: Cloudlet #1 return received
50.01: Broker: The number of finished Cloudlets is:2
51.51: Broker: Cloudlet #2 return received
52.51: Broker: The number of finished Cloudlets is:3
64.34333333333333: Broker: Cloudlet #3 return received
```

شکل ۲۱: خروجی های مربوط به cmd

```
20 Command Prompt
Initialising scenario: SMFT_LIKE
Initialising...
Running scenario: SMFT_LIKE
Starting CloudSim version 7.0
0.0: Datacenter_SMFT_LIKE is starting...
0.0: Broker is starting...
Entities started.
0.0: Broker: Cloud Resource List received with 1 datacenter(s)
0.0: Broker: Trying to Create Vm #0 in Datacenter_SMFT_LIKE
0.0: Broker: Trying to Create Vm #1 in Datacenter_SMFT_LIKE
0.0: Broker: Trying to Create Vm #2 in Datacenter_SMFT_LIKE
0.0: Datacenter_SMFT_LIKE_guestAllocator: Vm #0 has been allocated to Host #0
0.0: Datacenter_SMFT_LIKE_guestAllocator: Vm #1 has been allocated to Host #1
0.0: Datacenter_SMFT_LIKE_guestAllocator: Vm #2 has been allocated to Host #0
0.0: Datacenter_SMFT_LIKE_guestAllocator: Vm #3 has been allocated to Host #1
0.01: Broker: Vm #0 has been created in Datacenter #2, Host #0
0.01: Broker: Vm #1 has been created in Datacenter #2, Host #1
0.01: Broker: Vm #2 has been created in Datacenter #2, Host #0
0.01: Broker: Vm #3 has been created in Datacenter #2, Host #1
0.01: Broker: Sending Cloudlet #0 to Vm #0
0.01: Broker: Sending Cloudlet #1 to Vm #1
0.01: Broker: Sending Cloudlet #2 to Vm #2
0.01: Broker: Sending Cloudlet #3 to Vm #3
0.01: Broker: Sending Cloudlet #4 to Vm #0
0.01: Broker: Sending Cloudlet #5 to Vm #1
0.01: Broker: Sending Cloudlet #6 to Vm #2
0.01: Broker: Sending Cloudlet #7 to Vm #3
0.01: Broker: Sending Cloudlet #8 to Vm #0
0.01: Broker: Sending Cloudlet #9 to Vm #1
0.01: Broker: Sending Cloudlet #10 to Vm #2
0.01: Broker: Sending Cloudlet #11 to Vm #3
0.01: Broker: Sending Cloudlet #12 to Vm #0
0.01: Broker: Sending Cloudlet #13 to Vm #1
0.01: Broker: Sending Cloudlet #14 to Vm #2
0.01: Broker: Sending Cloudlet #15 to Vm #3
0.01: Broker: Sending Cloudlet #16 to Vm #0
0.01: Broker: Sending Cloudlet #17 to Vm #1
0.01: Broker: Sending Cloudlet #18 to Vm #2
0.01: Broker: Sending Cloudlet #19 to Vm #3
41.67666666666666: Broker: Cloudlet #0 return received
41.67666666666666: Broker: The number of finished Cloudlets is:1
50.01: Broker: Cloudlet #1 return received
50.01: Broker: The number of finished Cloudlets is:2
51.51: Broker: Cloudlet #2 return received
52.51: Broker: The number of finished Cloudlets is:3
64.34333333333333: Broker: Cloudlet #3 return received
```

شکل ۲۲: خروجی های مربوط به cmd

```

C:\> Command Prompt
0.01: Broker: Sending Cloudlet #11 to Vm #1
0.01: Broker: Sending Cloudlet #14 to Vm #2
0.01: Broker: Sending Cloudlet #12 to Vm #3
0.01: Broker: Sending Cloudlet #10 to Vm #0
0.01: Broker: Sending Cloudlet #17 to Vm #1
0.01: Broker: Sending Cloudlet #18 to Vm #2
0.01: Broker: Sending Cloudlet #19 to Vm #3
11.676666666666666: Broker: Cloudlet #0 return received
11.676666666666666: Broker: The number of finished Cloudlets is:1
00.01: Broker: Cloudlet #1 return received
00.01: Broker: The number of finished Cloudlets is:2
02.51: Broker: Cloudlet #2 return received
02.51: Broker: The number of finished Cloudlets is:3
04.343333333333333: Broker: Cloudlet #4 return received
04.343333333333333: Broker: The number of finished Cloudlets is:4
06.843333333333333: Broker: Cloudlet #5 return received
06.843333333333333: Broker: The number of finished Cloudlets is:5
07.676666666666666: Broker: Cloudlet #6 return received
07.676666666666666: Broker: The number of finished Cloudlets is:6
08.843333333333333: Broker: Cloudlet #8 return received
08.843333333333333: Broker: The number of finished Cloudlets is:7
10.343333333333333: Broker: Cloudlet #9 return received
10.343333333333333: Broker: The number of finished Cloudlets is:8
14.176666666666666: Broker: Cloudlet #7 return received
14.176666666666666: Broker: The number of finished Cloudlets is:9
06.676666666666666: Broker: Cloudlet #9 return received
06.676666666666666: Broker: The number of finished Cloudlets is:10
101.676666666666666: Broker: Cloudlet #12 return received
101.676666666666666: Broker: The number of finished Cloudlets is:11
100.841666666666666: Broker: Cloudlet #10 return received
100.841666666666666: Broker: The number of finished Cloudlets is:12
100.343333333333333: Broker: Cloudlet #16 return received
100.343333333333333: Broker: The number of finished Cloudlets is:13
110.008333333333333: Broker: Cloudlet #13 return received
110.008333333333333: Broker: The number of finished Cloudlets is:14
114.175833333333333: Broker: Cloudlet #17 return received
114.175833333333333: Broker: The number of finished Cloudlets is:15
116.675000000000001: Broker: Cloudlet #17 return received
116.675000000000001: Broker: The number of finished Cloudlets is:16
119.175000000000001: Broker: Cloudlet #14 return received
119.175000000000001: Broker: The number of finished Cloudlets is:17
127.508333333333333: Broker: Cloudlet #18 return received
127.508333333333333: Broker: The number of finished Cloudlets is:18
127.508333333333333: Broker: Cloudlet #15 return received
127.508333333333333: Broker: The number of finished Cloudlets is:19
140.01: Broker: Cloudlet #19 return received
140.01: Broker: The number of finished Cloudlets is:20
140.01: Broker: All Cloudlets executed. Finishing...
140.01: Broker: Destroying Vm #0
140.01: Broker: Destroying Vm #1
140.01: Broker: Destroying Vm #2
140.01: Broker: Destroying Vm #3
140.01: Broker is shutting down...
140.01: Simulation. No more future events
140.01: CloudInformationService is shutting down...
140.01: CloudInformationService: Notify all Cloudlet entities for shutting down.
140.01: Datacenter_SWIFT_LIKE is shutting down...
140.01: Broker is shutting down...
Simulation completed.
Simulation completed.

===== OUTPUT_SWIFT_LIKE =====
CloudletID STATUS DatacenterID QMID CPUTime StartTime FinishTime
0 SUCCESS 2 0 41.67 0.01 41.68
1 SUCCESS 2 1 50.80 0.01 50.81
2 SUCCESS 2 2 62.30 0.01 62.31
3 SUCCESS 2 0 68.33 0.01 68.34
4 SUCCESS 2 1 70.83 0.01 70.84
5 SUCCESS 2 1 76.67 0.01 76.68
6 SUCCESS 2 2 85.83 0.01 85.84
7 SUCCESS 2 0 88.33 0.01 88.34
8 SUCCESS 2 1 88.17 0.01 88.18
9 SUCCESS 2 1 96.67 0.01 96.68
10 SUCCESS 2 0 101.67 0.01 101.68
11 SUCCESS 2 2 105.83 0.01 105.84
12 SUCCESS 2 0 108.33 0.01 108.34
13 SUCCESS 2 1 110.00 0.01 110.01
14 SUCCESS 2 1 114.17 0.01 114.18
15 SUCCESS 2 1 116.67 0.01 116.68
16 SUCCESS 2 2 119.17 0.01 119.18
17 SUCCESS 2 1 127.50 0.01 127.51
18 SUCCESS 2 1 127.50 0.01 127.51
19 SUCCESS 2 1 140.00 0.01 140.01

===== METRICS_SWIFT_LIKE =====
Number of Cloudlets: 20
Total CPU Time: 1905.8208
Average Finish Time: 95.3010
Makespan: 140.0000
throughput (cloudlets/unit time): 0.142857
SwiftSimulation finished!
All outputs saved in folder: simulation_output
C:\Users\VAH\Downloads\cloudsim-7.0.1\cloudsim-7.0.1\modules\cloudsim-examples>

```

شکل ۲۳: خروجی های مربوط به cmd

```

C:\> Command Prompt
140.01: Broker: Destroying Vm #0
140.01: Broker: Destroying Vm #1
140.01: Broker: Destroying Vm #2
140.01: Broker: Destroying Vm #3
140.01: Broker is shutting down...
140.01: Simulation. No more future events
140.01: CloudInformationService is shutting down...
140.01: CloudInformationService: Notify all Cloudlet entities for shutting down.
140.01: Datacenter_SWIFT_LIKE is shutting down...
140.01: Broker is shutting down...
Simulation completed.
Simulation completed.

===== OUTPUT_SWIFT_LIKE =====
CloudletID STATUS DatacenterID QMID CPUTime StartTime FinishTime
0 SUCCESS 2 0 41.67 0.01 41.68
1 SUCCESS 2 1 50.80 0.01 50.81
2 SUCCESS 2 2 62.30 0.01 62.31
3 SUCCESS 2 0 68.33 0.01 68.34
4 SUCCESS 2 1 70.83 0.01 70.84
5 SUCCESS 2 1 76.67 0.01 76.68
6 SUCCESS 2 2 85.83 0.01 85.84
7 SUCCESS 2 0 88.33 0.01 88.34
8 SUCCESS 2 1 88.17 0.01 88.18
9 SUCCESS 2 1 96.67 0.01 96.68
10 SUCCESS 2 0 101.67 0.01 101.68
11 SUCCESS 2 2 105.83 0.01 105.84
12 SUCCESS 2 0 108.33 0.01 108.34
13 SUCCESS 2 1 110.00 0.01 110.01
14 SUCCESS 2 1 114.17 0.01 114.18
15 SUCCESS 2 1 116.67 0.01 116.68
16 SUCCESS 2 2 119.17 0.01 119.18
17 SUCCESS 2 1 127.50 0.01 127.51
18 SUCCESS 2 1 127.50 0.01 127.51
19 SUCCESS 2 1 140.00 0.01 140.01

===== METRICS_SWIFT_LIKE =====
Number of Cloudlets: 20
Total CPU Time: 1905.8208
Average Finish Time: 95.3010
Makespan: 140.0000
throughput (cloudlets/unit time): 0.142857
SwiftSimulation finished!
All outputs saved in folder: simulation_output
C:\Users\VAH\Downloads\cloudsim-7.0.1\cloudsim-7.0.1\modules\cloudsim-examples>

```

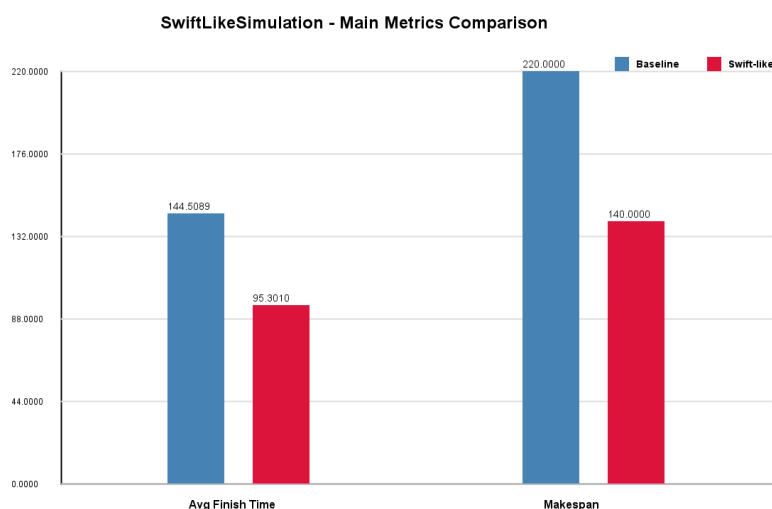
شکل ۲۴: خروجی های مربوط به cmd

تصاویر ثبت شده از محیط CMD نشان دهنده مراحل اجرای موفق برنامه شبیه سازی و تولید خروجی های نهایی هستند. در این تصاویر، ابتدا فایل جاوای مربوط به کلاس اصلی پروژه با استفاده از دستور javac کامپایل شده و سپس با استفاده از دستور java اجرا شده است. مشاهده اجرای بدون خطا در این مرحله بیانگر آن است که ساختار برنامه، وابستگی ها و منطق کلی پیاده سازی به درستی تنظیم شده اند.

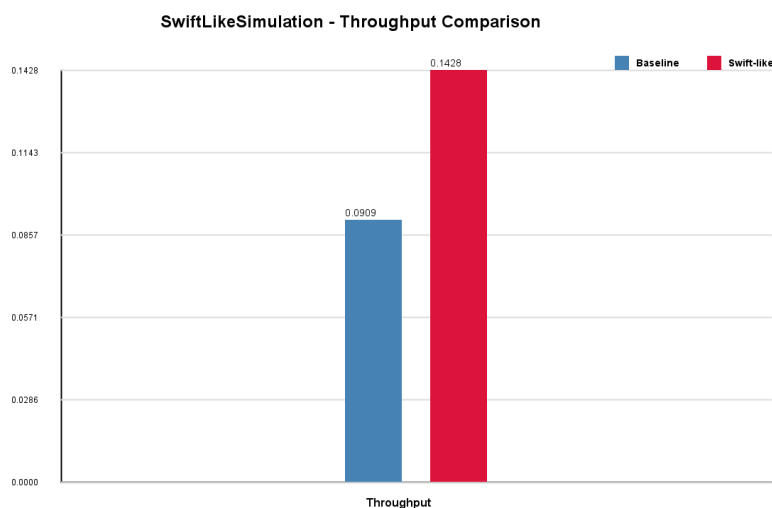
همچنین، در خروجی محیط CMD اطلاعات مربوط به اجرای دو سناریوی BASELINE و SWIFT_LIKE نمایش داده می شود. این بخش از خروجی اهمیت زیادی دارد، زیرا نشان می دهد که برنامه توانسته است هر دو حالت آزمایش را به صورت مستقل اجرا کرده و معیارهای عملکردی مورد نظر، از جمله زمان پایان وظایف، Makespan، Throughput و زمان کل مصرف CPU را محاسبه کند.

از دید مستندسازی، تصاویر CMD نقش مهمی در اثبات صحت اجرای پروژه دارند؛ زیرا علاوه بر نمایش روند اجرای شبیه سازی، بیان می کنند که داده های نهایی تحلیل شده در این گزارش، مستقیماً از اجرای واقعی برنامه به دست آمده اند و صرفاً مقادیر فرضی یا تئوریک نیستند.

۲۱.۰۰ توضیح نمودارهای خروجی



شکل ۲۵: نمودار مربوط به مقایسه معیار های اصلی



شکل ۲۶: نمودار مربوط به Throughput

برای تحلیل دیداری نتایج، دو نمودار مجزا از خروجی‌های شبیه‌سازی تهیه شده است. این نمودارها به درک بهتر تفاوت عملکرد میان دو سناریوی BASELINE و SWIFT_LIKE کمک می‌کنند.

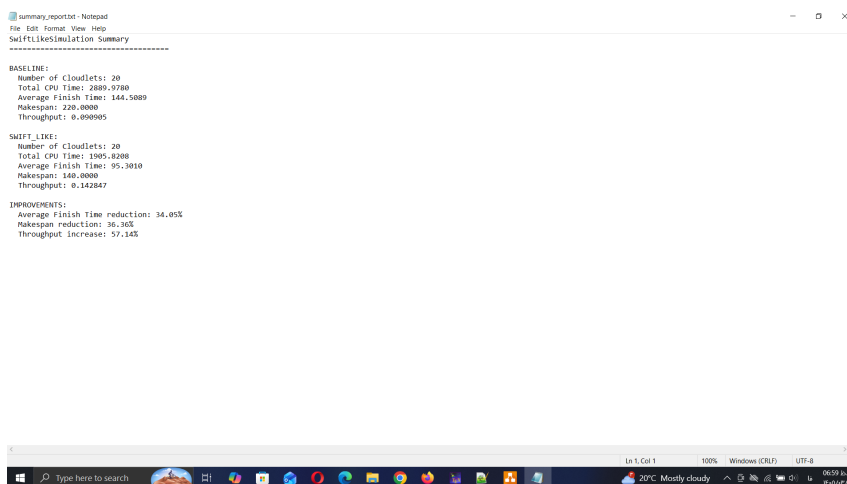
نمودار نخست، مقایسه معیارهای اصلی عملکرد شامل Average Finish Time، Total CPU Time و Makespan را میان دو سناریو نشان می‌دهد. در این نمودار، ارتفاع ستون‌های مربوط به سناریوی SWIFT_LIKE در هر سه معیار کمتر از سناریوی BASELINE است. این موضوع به صورت

بصری نشان می‌دهد که مدل پیشنهادی توانسته است هم زمان تکمیل وظایف را کاهش دهد، هم زمان کل اجرای بار کاری را کم کند، و هم استفاده کارآمدتری از زمان پردازشی CPU داشته باشد. به بیان دیگر، این نمودار نشان‌دهنده کاهش هزینه زمانی سیستم در اثر استفاده از رویکرد الهام‌گرفته از Swift است.

نمودار دوم به‌صورت اختصاصی معیار Throughput را در دو سناریو مقایسه می‌کند. در این نمودار، مقدار Throughput در حالت SWIFT_LIKE به‌طور قابل توجهی بیشتر از حالت پایه است. این افزایش نشان می‌دهد که در سناریوی پیشنهادی، تعداد بیشتری از وظایف در واحد زمان تکمیل شده‌اند. از آنجا که Throughput یکی از مهم‌ترین معیارهای ارزیابی کارایی در سامانه‌های ابری و مراکز داده است، این نمودار نقش مهمی در اثبات برتری عملکردی مدل پیشنهادی ایفا می‌کند.

در مجموع، این دو نمودار تصویری روشن از نتایج شبیه‌سازی ارائه می‌دهند و به‌خوبی نشان می‌دهند که رویکرد SWIFT_LIKE نسبت به حالت پایه، موجب بهبود هم‌زمان در شاخص‌های زمانی و نرخ پردازش شده است.

۲۲.۰۰ توضیح فایل‌های متنی و داده‌ای

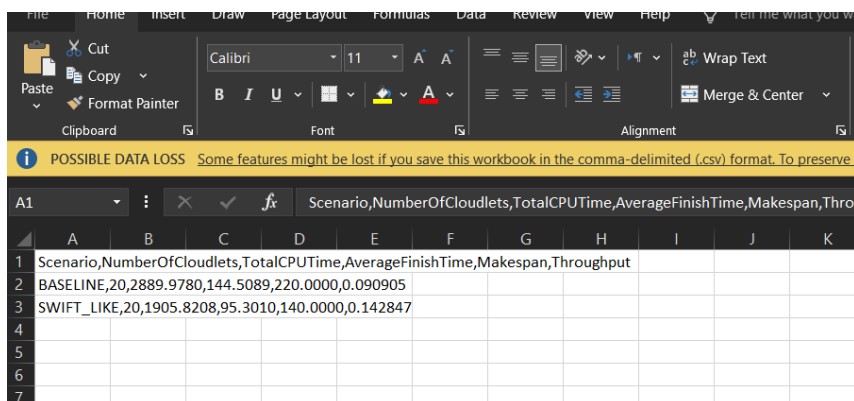


```
summary_report.txt
SwiftLikeSimulation Summary
-----
BASELINE:
Number of Cloudlets: 20
Total CPU Time: 2889.9780
Average Finish Time: 144.5089
Makespan: 220.0000
Throughput: 0.090905

SWIFT_LIKE:
Number of Cloudlets: 20
Total CPU Time: 1905.8208
Average Finish Time: 95.3010
Makespan: 140.0000
Throughput: 0.142847

IMPROVEMENTS:
Average Finish Time reduction: 34.05%
Makespan reduction: 36.36%
Throughput increase: 57.14%
```

شکل ۲۷: فایل summary_report.txt



Scenario	NumberOfCloudlets	TotalCPUTime	AverageFinishTime	Makespan	Throughput
BASELINE	20	2889.9780	144.5089	220.0000	0.090905
SWIFT_LIKE	20	1905.8208	95.3010	140.0000	0.142847

شکل ۲۸: فایل metrics_comparison.csv

علاوه بر خروجی‌های تصویری، نتایج شبیه‌سازی در دو فایل داده‌ای نیز ذخیره شده‌اند که شامل فایل summary_report.txt و فایل metrics_comparison.csv هستند. این دو فایل نقش مهمی در مستندسازی، تحلیل عددی و ارجاع‌پذیری نتایج دارند.

فایل summary_report.txt یک گزارش متنی خلاصه از اجرای شبیه‌سازی است که در آن نتایج نهایی دو سناریوی BASELINE و SWIFT_LIKE به صورت خوانا و ساخت‌یافته ارائه شده‌اند. این فایل برای مرور سریع نتایج بسیار مناسب است و به پژوهشگر کمک می‌کند بدون نیاز به پردازش مجدد داده‌ها، تصویری کلی از عملکرد دو سناریو به دست آورد. در این گزارش متنی، تعداد Cloudlet، زمان کل مصرف CPU، میانگین زمان پایان، Makespan و Throughput برای هر دو حالت ثبت شده‌اند.

در مقابل، فایل metrics_comparison.csv داده‌های عددی نتایج را در قالب جدولی و ساخت‌یافته ذخیره می‌کند. این فایل برای تحلیل دقیق‌تر، ترسیم نمودارها، تهیه جداول مقایسه‌ای و استفاده از ابزارهای آماری بسیار مفید است. مزیت اصلی قالب CSV در این است که داده‌ها به راحتی قابل انتقال به نرم‌افزارهایی مانند Excel، Python، MATLAB و سایر ابزارهای تحلیل داده هستند. بنابراین، این فایل مبنای اصلی تحلیل کمی نتایج و استخراج درصد بهبود هر معیار در این پروژه محسوب می‌شود.

وجود هم‌زمان فایل متنی و فایل CSV باعث شده است که خروجی‌های پروژه هم از نظر خوانایی انسانی و هم از نظر قابلیت پردازش ماشینی، به صورت کامل و مناسب مستند شوند.

۲۳.۰۰ لینک‌دهی و خروجی‌ها در گیت هاب

لینک فایل‌های مربوط به پروژه اعم از کد، خروجی‌ها و گزارش، در گیت هاب به صورت زیر می‌باشد:

<https://github.com/Aysooda-Hh/swift-cloudsim-simulation>

جدول مقایسه نتایج دو سناریو

جدول ۱ مقادیر نهایی معیارهای عملکرد را برای دو سناریوی BASELINE و SWIFT_LIKE نشان می‌دهد.

جدول ۱: مقایسه معیارهای عملکرد در دو سناریوی BASELINE و SWIFT_LIKE

Metric	BASELINE	SWIFT_LIKE
NumberOfCloudlets	20	20
TotalCPUTime	2889.9780	1905.8208
AverageFinishTime	144.5089	95.3010
Makespan	220.0000	140.0000
Throughput	0.090905	0.142847

۲۴.۰۰ تحلیل جدول مقایسه

همان‌طور که در جدول ۱ مشاهده می‌شود، تعداد Cloudlet‌ها در هر دو سناریو برابر با ۲۰ است؛ بنابراین، شرایط آزمایش از نظر حجم بار کاری یکسان بوده و مقایسه انجام‌شده معتبر است.

بر اساس داده‌های جدول، مقدار Average Finish Time از ۱۴۴.۵۰۸۹ در حالت BASELINE به ۹۵.۳۰۱۰ در حالت SWIFT_LIKE کاهش یافته است. این تغییر نشان‌دهنده کاهش تقریبی ۳۴.۰۵ درصدی در میانگین زمان پایان وظایف است. چنین کاهش به این معناست که وظایف در مدل پیشنهادی سریع‌تر تکمیل شده‌اند و زمان انتظار کلی سیستم کاهش یافته است.

همچنین، معیار Makespan از ۲۲۰.۰۰۰۰ به ۱۴۰.۰۰۰۰ کاهش یافته است که معادل ۳۶.۳۶ درصد بهبود است. این معیار نشان می‌دهد که کل مجموعه وظایف در سناریوی پیشنهادی در زمان کوتاه‌تری به پایان رسیده‌اند. کاهش Makespan یکی از شاخص‌های مهم در افزایش بهره‌وری مراکز داده محسوب می‌شود.

در مورد Throughput، مقدار این شاخص از ۰.۰۹۰۹۰۵ در سناریوی پایه به ۰.۱۴۲۸۴۷ در سناریوی SWIFT_LIKE افزایش یافته است. این افزایش تقریباً ۵۷.۱۴ درصدی نشان می‌دهد که تعداد بیشتری از وظایف در واحد زمان پردازش و تکمیل شده‌اند. بنابراین، سیستم در سناریوی پیشنهادی توان عملیاتی بالاتری از خود نشان داده است.

علاوه بر این، مقدار Total CPU Time نیز از ۲۸۸۹.۹۷۸۰ به ۱۹۰۵.۸۲۰۸ کاهش یافته است. این نتیجه بیانگر استفاده کارآمدتر از منابع پردازشی و کاهش هزینه زمانی اجرای وظایف در مدل پیشنهادی است.

در مجموع، جدول مقایسه نتایج به‌روشنی نشان می‌دهد که سناریوی SWIFT_LIKE در تمامی



معیارهای کلیدی عملکرد، بهتر از سناریوی BASELINE عمل کرده است. این موضوع مؤید آن است که شبیه‌سازی مفهومی الگوریتم الهام گرفته از Swift توانسته است بهبود قابل توجهی در کارایی کلی سیستم ایجاد کند.

مزایای پیاده سازی با cloudsim

استفاده از CloudSim در این پروژه مزایای قابل توجهی داشته است که آن را به بستری مناسب برای شبیه سازی مفهومی الگوریتم کنترل ازدحام الهام گرفته از Swift تبدیل می کند. نخستین مزیت CloudSim، فراهم کردن یک محیط شبیه سازی سبک، منعطف و قابل کنترل برای مدل سازی اجزای اصلی مرکز داده است. در این محیط، پژوهشگر می تواند بدون نیاز به پیاده سازی واقعی زیرساخت سخت افزاری یا شبکه ای، رفتار ماشین های مجازی، میزبان ها، وظایف پردازشی و سیاست های تخصیص منابع را مورد بررسی قرار دهد.

مزیت دوم، امکان تعریف و مقایسه سناریوهای مختلف در شرایط یکسان است. در این پروژه، دو سناریوی BASELINE و SWIFT_LIKE با تعداد یکسانی از Cloudlet ها و تحت یک بستر شبیه سازی مشترک اجرا شدند. این ویژگی باعث شد که ارزیابی عملکرد دو حالت، به شکل منصفانه و مبتنی بر شاخص های کمی انجام گیرد. در نتیجه، تفاوت مشاهده شده در معیارهایی مانند Average Finish Time، Makespan، Throughput و Total CPU Time را می توان به منطق کنترلی و سیاست زمان بندی اعمال شده نسبت داد، نه به تفاوت در شرایط اجرای آزمایش.

از دیگر مزایای مهم CloudSim، قابلیت استخراج آسان معیارهای عملکردی و تولید خروجی های قابل تحلیل است. این بستر امکان ثبت داده های عددی، تولید گزارش های متنی، و تهیه فایل های ساخت یافته مانند CSV را فراهم می سازد. چنین قابلیت هایی در این پروژه بسیار ارزشمند بوده است؛ زیرا تحلیل نتایج، ترسیم نمودارها، تهیه جداول مقایسه ای و مستندسازی نهایی پژوهش را ساده تر و دقیق تر کرده است.

همچنین، CloudSim این امکان را فراهم می کند که پژوهشگر بر جنبه های سطح سیستم و اثرات کلان یک ایده متمرکز شود. با توجه به اینکه الگوریتم Swift در اصل برای کنترل ازدحام در سطح شبکه و مبتنی بر پارامترهایی مانند RTT طراحی شده است، بازسازی کامل آن در یک شبیه ساز رویدادگسسته سطح رایانش ابری مانند CloudSim به صورت مستقیم امکان پذیر نیست. با این حال، CloudSim بستری مناسب برای بررسی اثرات مفهومی این رویکرد بر شاخص های کلان عملکرد، مانند کاهش تأخیر، بهبود زمان تکمیل وظایف و افزایش توان عملیاتی، فراهم می کند. این ویژگی دقیقاً با هدف این پروژه هم راستا بوده است.

علاوه بر موارد فوق، سادگی در پیاده سازی، قابلیت توسعه، و دسترسی گسترده به نمونه کدها و مستندات، از دیگر مزیت های CloudSim در این پروژه به شمار می روند. این ویژگی ها سبب شدند که طراحی سناریوی پیشنهادی، اجرای آزمایش ها و تفسیر نتایج با سرعت و دقت بیشتری انجام شود.

در مجموع، می توان گفت که CloudSim در این پروژه نه تنها به عنوان یک ابزار شبیه سازی، بلکه به عنوان یک بستر پژوهشی مؤثر برای ارزیابی مفهومی ایده های مرتبط با بهبود عملکرد مراکز داده عمل کرده است. استفاده از این محیط باعث شد که بدون نیاز به پیاده سازی پیچیده در سطح



پروتکل های واقعی شبکه، بتوان اثرات عملکردی رویکرد الهام گرفته از Swift را به صورت کمی بررسی و تحلیل کرد.

جمع بندی

در این پروژه، یک مدل مفهومی از الگوریتم Swift در محیط CloudSim پیاده‌سازی و ارزیابی شد. دو سناریو شامل حالت پایه و حالت Swift-like تعریف گردیدند و عملکرد آن‌ها بر اساس سه شاخص اصلی مورد مقایسه قرار گرفت.

نتایج شبیه‌سازی نشان داد که در سناریوی Swift-like :

۱. میانگین زمان تکمیل وظایف کاهش یافت،

۲. طول کل اجرای سیستم (Makespan) کمتر شد،

۳. و نرخ خروجی سیستم افزایش پیدا کرد.

این نتایج بیانگر آن است که کاهش تأخیر و بهبود مدیریت منابع می‌تواند تأثیر مستقیمی بر کارایی کلان مراکز داده داشته باشد.

نتیجه‌گیری

این مطالعه نشان داد که حتی یک مدل سطح بالا و مفهومی از یک الگوریتم مبتنی بر تأخیر مانند Swift می‌تواند منجر به بهبود شاخص‌های عملکردی در محیط شبیه‌سازی شود. هرچند این پیاده‌سازی جایگزین بازسازی کامل الگوریتم در سطح TCP یا بسته‌های شبکه نیست، اما نشان می‌دهد که اصول طراحی Swift، یعنی تمرکز بر کاهش تأخیر صف و واکنش سریع به شرایط ازدحام، می‌تواند در سطح سیستم نیز آثار مثبتی داشته باشد.

از جمله محدودیت‌های این پژوهش می‌توان به نبود مدل‌سازی دقیق RTT، عدم وجود صف شبکه واقعی و عدم پیاده‌سازی مکانیزم‌های کنترلی سطح TCP اشاره کرد. در تحقیقات آینده، می‌توان با استفاده از شبیه‌سازهای سطح شبکه مانند ns-3 یا پیاده‌سازی در هسته سیستم‌عامل، بازسازی دقیق‌تری از الگوریتم Swift انجام داد.

به طور کلی، نتایج این پروژه نشان می‌دهد که رویکردهای مبتنی بر تأخیر می‌توانند گزینه‌ای مؤثر برای بهبود کارایی مراکز داده باشند و بررسی عمیق‌تر آن‌ها در محیط‌های دقیق‌تر شبکه‌ای توصیه می‌شود.



فهرست منابع

۱. [Swift:DelayisSimpleandEffectiveforCongestionControlintheDatacenter](#)

۲. [CloudSimOfficialGitHubRepository](#)